FULL STACK DEVELOPMENT

Final Assessment | Reference Answer Script

Based on the supplied assessment. Includes all 25 multiple-choice answers, 10 short answers, five code-writing solutions and capstone documentation.

Candidate name: _____

Student ID: _____ Batch: _____

Date: _____

Use and completion status

This is reference and practice material. The assessment states that Sections A-C are closed book and the Section D lab prohibits AI tools. Review the course rules before use and be ready to explain the work yourself.

The accompanying package includes source code, practical examples, Laravel migrations and project documentation. Public deployment, a remote repository URL, candidate details, the live demonstration and viva remain to be completed by the candidate.

Package guide

output/Answer_Script.md - editable version of these answers

practicals/C1-C5 - source examples for Section C

portal/ - full Laravel and Bootstrap capstone

docs/ - setup handoff, submission checklist, deployment and viva notes

Section A - Multiple choice

25 questions | 25 marks

Q1. A - A website mainly presents content; a web application supports interactive data processing.

Q2. C - <h1> is the highest-level heading and is the intended answer; CSS can change its visual size.

Q3. B - The alt attribute supplies alternative text.

Q4. D - #main targets the element with id="main".

Q5. B - The order is content, padding, border, margin.

Q6. C - .col-md-6 uses six of twelve columns from the medium breakpoint.

Q7. A - const prevents reassignment of the variable binding.

Q8. D - The statement replaces the contents of the existing demo element with Hi.

Q9. B - A domain name and web hosting is the best offered option. A hosting-provided subdomain also works; buying a custom domain is optional.

Q10. C - PHP variable names begin with $.

Q11. A - XAMPP bundles Apache, MariaDB/MySQL, PHP and Perl.

Q12. D - SELECT retrieves rows.

Q13. B - $_POST holds ordinary form fields sent using POST.

Q14. C - A primary key uniquely identifies each row.

Q15. A - JOIN combines related rows.

Q16. B - Laravel follows the Model-View-Controller pattern.

Q17. D - php artisan make:controller UserController

Q18. C - password_hash() creates a password hash; password_verify() checks it.

Q19. B - POST normally creates a resource.

Q20. C - 404 means not found.

Q21. A - Express.js is a Node.js web framework.

Q22. D - useState provides state and a setter that can trigger a re-render.

Q23. B - Angular applications are primarily written in TypeScript.

Q24. A - React Native supports Android and iOS from a shared codebase.

Q25. C - Kotlin and Android Studio are used for native Android development.

Section B - Short answers

10 questions | 30 marks

B1. Website and web application

A website primarily presents information, such as a company brochure site. A web application lets users perform tasks and process data, such as Gmail sending and organising messages. Both use web technologies; many modern sites include application features.

B2. CSS box model

An element consists of content, padding, border and margin, from inside to outside. Padding is space between the content and border; margin is space outside the border. With box-sizing: border-box, the declared width includes the content, padding and border, but excludes margin.

B3. Bootstrap grid

Bootstrap uses containers, rows and a 12-column grid. Column classes specify how much of a row an element occupies. For example, col-12 col-md-4 stacks items on small screens and places three equal columns on medium screens and above. Breakpoint classes adapt the layout to available width.

B4. var, let and const

var is function-scoped, permits redeclaration and reassignment, and is hoisted with an initial value of undefined; avoid it in new block-scoped code. let is block-scoped and suits values that change. const is block-scoped, must be initialised and cannot be reassigned; use it by default when the binding stays the same. Properties of a const object can still change.

B5. GET and POST

GET places form data in the URL query string and is suitable for retrieval, searches and shareable filters. POST sends fields in the request body and is suitable for submissions that change data. Use POST for login so credentials do not appear in the URL. HTTPS is still required: POST alone does not encrypt data.

Section B - Short answers (continued)

B6. Primary and foreign keys

A primary key is a unique, non-null identifier for a row, such as courses.id. A foreign key references a key in another table, such as registrations.course_id referencing courses.id. Together they link related records, enforce referential integrity and reduce duplicated data.

B7. SQL injection

SQL injection occurs when untrusted input changes the structure of a database query. Prevent it by using PDO prepared statements with bound parameters, or Laravel Eloquent/query-builder parameter binding instead of concatenating input into SQL. Also validate input and allow-list dynamic column names; placeholders cannot bind SQL identifiers.

B8. Laravel MVC

The Model represents data and relationships, usually through Eloquent. The View renders the user interface using Blade. The Controller handles a request, validates input, works with models and returns a view or response. The route file maps an HTTP method and URL to the relevant controller action.

B9. REST API and CRUD

A REST API exposes resources through URLs using a stateless request/response style. GET reads resources, POST creates them, PUT replaces or updates a resource, and DELETE removes it. JSON is the usual response format. For example, GET /api/courses reads courses, with HTTP status codes describing the outcome.

B10. Native and cross-platform mobile apps

Native apps target a particular platform using its languages and SDKs, for example Kotlin with Android Studio for Android. Cross-platform apps share much of their code across platforms, for example React Native for Android and iOS. Native development offers direct platform integration but usually requires separate codebases; cross-platform development reduces duplication but may still need platform-specific modules.

Section C - Code writing

5 questions | 25 marks. Long lines may wrap visually; use the accompanying source files when running the examples.

C1. Responsive Bootstrap layout

col-12 stacks on mobile; col-md-4 creates three equal columns at medium widths and above.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Three responsive cards</title>
  <link rel="stylesheet" href="../../portal/public/assets/bootstrap.min.css">
</head>
<body>
<section class="container py-5" aria-label="Courses">
  <div class="row g-4">
    <div class="col-12 col-md-4">
      <div class="card h-100"><div class="card-body">
        <h2 class="card-title h5">HTML</h2>
        <p class="card-text">Learn to structure accessible web pages.</p>
        <a href="#html" class="btn btn-primary">Learn HTML</a>
      </div></div>
    </div>
    <div class="col-12 col-md-4">
      <div class="card h-100"><div class="card-body">
        <h2 class="card-title h5">CSS</h2>
        <p class="card-text">Style pages and build responsive layouts.</p>
        <a href="#css" class="btn btn-primary">Learn CSS</a>
      </div></div>
    </div>
    <div class="col-12 col-md-4">
      <div class="card h-100"><div class="card-body">
        <h2 class="card-title h5">JavaScript</h2>
        <p class="card-text">Add interactive behaviour to your pages.</p>
        <a href="#javascript" class="btn btn-primary">Learn JavaScript</a>
      </div></div>
    </div>
  </div>
</section>
</body>
</html>
```

C2. JavaScript validation

Attach this function to the supplied form. Return false blocks submission and textContent displays errors safely.

```
function validateForm() {
  const name = document.getElementById('name').value.trim();
  const email = document.getElementById('email').value.trim();
  const password = document.getElementById('password').value;
  const error = document.getElementById('error');
  const messages = [];
  if (name === '') messages.push('Name is required.');
  if (!email.includes('@')) messages.push('Email must contain @.');
  if (password.length < 8) {
    messages.push('Password must contain at least 8 characters.');
  }
  error.textContent = messages.join(' ');
  return messages.length === 0;
}
```

C3. PHP and MySQL with PDO

The table is assumed to exist. Use the supplied SQL and HTML form to run the example; configure database credentials first.

```
<?php
// Set DB_USER and DB_PASS in your environment before running.
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    header('Allow: POST');
    exit('Please submit the POST form.');
}
$name = trim($_POST['name'] ?? '');
$email = trim($_POST['email'] ?? '');
if ($name === '' || strlen($name) > 120 ||
    !filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($email) > 254) {
    http_response_code(422);
    exit('Enter a name (maximum 120 characters) and a valid email.');
}
try {
    $pdo = new PDO(
        'mysql:host='.(getenv('DB_HOST') ?: '127.0.0.1').
        ';port='.(getenv('DB_PORT') ?: '3306').
        ';dbname=college;charset=utf8mb4',
        getenv('DB_USER') ?: 'root',
        getenv('DB_PASS') ?: '',
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
         PDO::ATTR_EMULATE_PREPARES => false]
    );
    $stmt = $pdo->prepare(
        'INSERT INTO students (name, email) VALUES (:name, :email)'
    );
    $stmt->execute(['name' => $name, 'email' => $email]);
    echo 'Student added successfully.';
} catch (PDOException $e) {
    error_log($e->getMessage());
    http_response_code(500);
    echo 'Database connection or insert failed. Please try again later.';
}
```

C4. Laravel products list

(a) Route - routes/web.php

```
<?php
// Add to routes/web.php in an existing Laravel application.
use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;
Route::get('/products', [ProductController::class, 'index']);
```

(b) Controller method - ProductController.php

```
<?php
namespace App\Http\Controllers;
use App\Models\Product;
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }
}
```

(c) Blade view - products/index.blade.php

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>Products</title></head>
<body>
<h1>Products</h1>
<ul>
@forelse ($products as $product)
  <li>{{ $product->name }} - {{ number_format($product->price, 2) }}</li>
@empty
  <li>No products available.</li>
@endforelse
</ul>
</body></html>
```

C5. Express endpoint and React component

(a) Express application - server.js

```
import express from 'express';

const app = express();
app.get('/api/products', (req, res) => {
  res.json([
    { id: 1, name: 'Keyboard', price: 2500 },
    { id: 2, name: 'Mouse', price: 1500 }
  ]);
});
app.listen(3001, '127.0.0.1', () => {
  console.log('Products API: http://127.0.0.1:3001/api/products');
});
```

(b) React component - Products.jsx

```
import React, { useEffect, useState } from 'react';

export default function Products() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    const controller = new AbortController();
    async function loadProducts() {
      try {
        const response = await fetch('/api/products', {
          signal: controller.signal
        });
        if (!response.ok) throw new Error('Could not load products.');
        setProducts(await response.json());
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }
    loadProducts();
    return () => controller.abort();
  }, []);

  if (loading) return <p>Loading products...</p>;
  if (error) return <p role="alert">{error}</p>;
  return (
    <ul>
      {products.map(product => <li key={product.id}>{product.name}</li>)}
    </ul>
  );
}
```

Section D - Practical capstone

Course Registration Portal | 20 marks

Implementation

Laravel 13 and PHP provide the backend. Blade, Bootstrap 5 and JavaScript provide the responsive frontend. MySQL-compatible migrations define users, courses and registrations. The portal does not require a separate React application.

Database design

courses.id and registrations.id are primary keys. registrations.course_id references courses.id with cascading delete. The composite unique key (course_id, email) prevents duplicate registrations for the same course. The users table has an is_admin flag. Migrations are the authoritative database deliverable.

Administrator workflows

Administrators sign in, list courses, add and edit validated course data, delete courses and view registrations. Both authentication and admin middleware protect management routes. The portal:admin command creates an administrator interactively, with no hard-coded password.

Visitor workflows

Visitors browse responsive course cards, filter them with live search and register using name, email and phone. Input is validated on the server; email is normalised to lowercase. Duplicate registration is blocked by both validation and a database unique constraint, including simultaneous requests.

REST endpoint

GET /api/courses returns HTTP 200 and a JSON array containing all courses. Only course data is exposed; registration contact details remain in the protected admin view.

Security

Passwords are hashed. Write forms include Laravel CSRF tokens. Blade escapes values. Eloquent binds query parameters. Login renews the session and logout invalidates it. Login, registrations and the API are rate limited.

How to run

Install Composer dependencies, copy .env.example to .env, create a MySQL database and set its credentials, run php artisan key:generate, php artisan migrate --seed, php artisan portal:admin, and php artisan serve. Detailed instructions and test results are supplied with the project.

Publishing and remaining evidence

Deploy the portal to Laravel-compatible hosting and supply its real URL. Push the code to your Git repository and supply that link. Complete candidate details and perform the five-minute demo and viva. No public deployment or remote repository has been created by this package.

Additional assignments

No earlier practical or assignment briefs were supplied. The package covers the tasks stated in this assessment; include any additional work your instructor separately requested.